Public Health Colosseum

Platform Manual

Release 1.0 — "Neo"

An AI-ready opportunity and community intelligence platform
for public-health students, professionals and researchers

Release	1.0.0 — "Neo"
Status	Production-ready
Date	4 October 2026 (revised 29 October 2026)
Surfaces	WhatsApp, Web

Powered by SCIAT SOLUTIONS

Contents

1 Introduction **4**
- 1.1 What this platform is . . . 4
- 1.2 What it deliberately does not do . . . 4
- 1.3 Current status . . . 4

2 Architecture **6**
- 2.1 Overview . . . 6
- 2.2 Design principles . . . 7
- 2.3 Request lifecycle . . . 7
- 2.4 Project layout . . . 7

3 Requirements **9**

4 Installation **10**
- 4.1 Obtain and install . . . 10
- 4.2 Verify the toolchain . . . 10

5 Configuration **11**
- 5.1 Where configuration lives . . . 11
- 5.2 Validation behaviour . . . 11
- 5.3 Complete variable reference . . . 11
- 5.4 A complete example . . . 13

6 Database **14**
- 6.1 Option A: rootless local server . . . 14
- 6.2 Option B: your own PostgreSQL . . . 14
- 6.3 Creating the schema . . . 14
- 6.4 Schema . . . 14

7 Running the platform **16**
- 7.1 Commands . . . 16
- 7.2 Startup order . . . 16
- 7.3 Searching from the terminal . . . 16

8 Integration: WhatsApp **17**
- 8.1 Ownership model . . . 17
- 8.2 Pairing with a QR code . . . 17
- 8.3 Pairing with an eight-character code . . . 17
- 8.4 Confirming the connection . . . 17
- 8.5 Who can use the bot . . . 17
- 8.6 Reconnection and automatic re-pairing . . . 18

9 Integration: the web interface **19**
- 9.1 Using it . . . 19
- 9.2 HTTP endpoints . . . 19
- 9.3 Example . . . 19
- 9.4 Rate limiting . . . 20
- 9.5 Security headers . . . 20

10 Integration: opportunity sources **21**
10.1 How sources work . . . 21
10.2 Registered sources . . . 21
10.3 Employer career boards (recommended) . . . 21
10.4 Enabling ReliefWeb . . . 22
10.5 On LinkedIn and similar platforms . . . 22
10.6 Adding a source . . . 23

11 Integration: AI providers **24**
11.1 What AI is used for . . . 24
11.2 Gemini — the default . . . 24
11.3 Anthropic . . . 24
11.4 OpenAI . . . 25
11.5 Switching providers . . . 25
11.6 Guardrails . . . 25
11.7 The .ask command . . . 25

12 Integration: web search **26**
12.1 Why search is separate from the model . . . 26
12.2 Choosing a backend . . . 26
12.3 How the fallback behaves . . . 27
12.4 Verified example . . . 27

13 Command reference **28**
13.1 Opportunities . . . 28
13.2 Navigating results . . . 28
13.3 Group management . . . 28
13.4 Moderation . . . 29
13.5 Welcome messages . . . 30
13.6 System . . . 30
13.7 Locations . . . 30
13.8 Filters . . . 31

14 The opportunity pipeline **32**
14.1 Normalisation . . . 32
14.2 Classification . . . 32
14.3 De-duplication . . . 33
14.4 Ranking . . . 33
14.5 Verification lifecycle . . . 34

15 Security and privacy **35**

16 Development **36**
16.1 Adding a command . . . 36
16.2 Adding a source . . . 36
16.3 Testing . . . 36

17 Troubleshooting **37**

18 Roadmap **39**
18.1 What Neo includes . . . 39
18.2 Next . . . 39

A Operational reference **41**
A.1 Default ports 41
A.2 Operational limits 41
A.3 Further reading 41

CHAPTER 1

Introduction

1.1 What this platform is

The Public Health Colosseum is a global community where students, early professionals, researchers and practitioners learn, connect and unlock opportunities in public health. This platform is the infrastructure behind it.

It discovers opportunities from multiple independent sources, normalises them into a single trustworthy shape, removes duplicates, hides anything past its deadline, ranks what remains, and delivers the result through two surfaces:

- **WhatsApp** — 34 commands for members already in the community.
- **Web** — a public search page requiring no account, no login and no app.

Both surfaces call the same search engine, so an improvement to ranking or de-duplication reaches both at once.

The platform covers fourteen opportunity types, including jobs, internships, scholarships, fellowships, research posts, grants, conferences, training and events.

1.2 What it deliberately does not do

Stating this plainly is part of the product. A member deciding whether to spend a week on an application deserves to know what the system is actually claiming.

- **It does not assert that listings are genuine.** Opportunities are gathered from third-party sources. Unless a record is explicitly marked verified, it has not been independently checked. Every rendered result says so and links to the official page.
- **It does not invent missing information.** A field a source did not state is stored as null and rendered as "not stated". An unknown deadline is never shown as "no deadline", because those are different facts.
- **It does not scrape sites that prohibit it.** See Section 10.5.
- **It does not ban or block members automatically.** Moderation is designed as *AI recommendation → risk score → administrator review.*

1.3 Current status

Release 1.0 "Neo" is complete, verified and in use. It covers opportunity discovery across six sources, two delivery surfaces, group administration, an AI assistant and a live web-search fallback. Later phases are specified in Chapter 18 but not built.

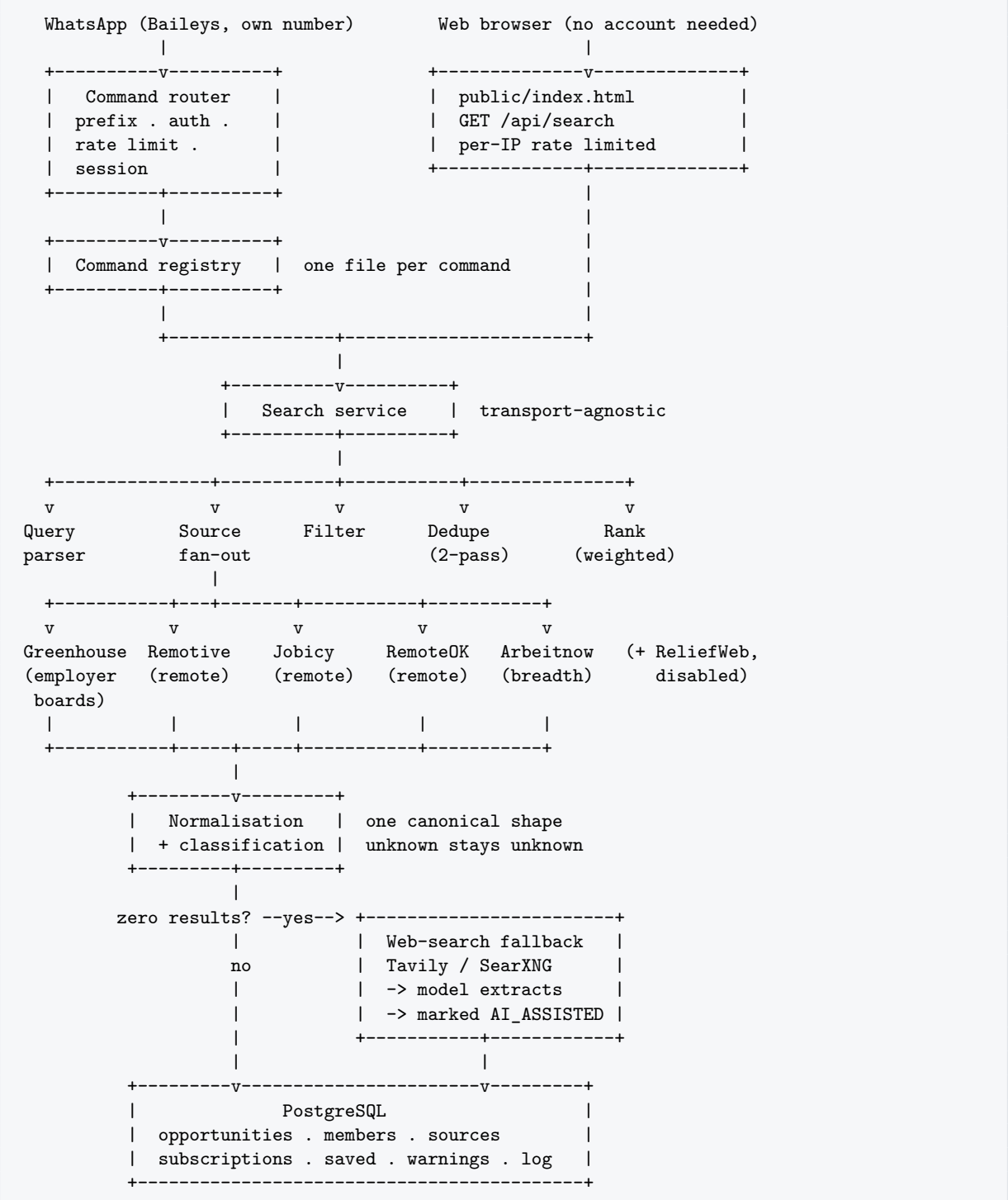
This manual documents only what exists today. Anything unverified is labelled as such — there are two such caveats, and both say so plainly where they appear.

Measure	Value
Source files (TypeScript, excluding tests)	64
Lines of source	8,432
Unit tests	133, all passing
Typecheck	Clean (`tsc --noEmit`)
Build	Clean
Commands	34 (63 names including aliases)
Opportunity sources	6 registered, 5 active
AI providers	Gemini (default), Anthropic; OpenAI planned
Web-search backends	Tavily, self-hosted SearXNG, provider grounding
Database tables	8 tables, 9 enumerated types

CHAPTER 2

Architecture

2.1 Overview



2.2 Design principles

Principle	How it is enforced
Provider independence	No business logic imports a vendor SDK. AI sits behind an interface selected by configuration.
Source independence	No code names a specific job board outside its own connector file. Adding a source touches two files.
Transport independence	The search service takes a query and returns scored results. It has no knowledge of WhatsApp or HTTP.
Unknown stays unknown	Unstated fields are null and render as “not stated”.
Graceful degradation	A failing source, missing AI key or thrown handler degrades one thing, and the user is told.
Honest verification	Nothing is presented as verified because it appeared in a feed.
Minimal personal data	A member record is a WhatsApp JID plus whatever the member chose to share.

2.3 Request lifecycle

```
inbound message
  |
  +- parseInvocation    prefix check; ordinary conversation ignored
  +- touchMember        upsert by JID, refresh lastSeenAt, re-sync admin flag
  +- isBlocked?         drop
  +- adminOnly?         check ADMIN_NUMBERS -> non-admins get "unknown command"
  +- rate limit         persisted token bucket, unless exempt
  |
  +- handler
       +- parseQuery    filters extracted, consumed, removed from free text
       +- react         immediate feedback before the slow part
       +- fanOut        connectors in parallel, each isolated and timed out
       +- applyFilters  hard requirements (type, country, remote, recency)
       +- dedupe        fingerprint pass, then same-org near-duplicate pass
       +- rank          weighted score plus relevance quality gate
       +- setSession    remembered for .next / .prev / .details
       +- reply         rendered page, attribution, degradation warning
       +- logCommand    command name and normalised terms only
```

Persistence is deliberately off this path. Discovered opportunities are written in the background and tracked, so shutdown waits for those writes rather than aborting them.

2.4 Project layout

```
src/
  index.ts              entry point, startup order, graceful shutdown
  cli/search.ts         terminal search harness
  config/env.ts         validated environment; the only reader of process.env
  core/                 logger (JID redaction), TTL cache, text utilities
  domain/               Opportunity model, classification, normalisation
  db/                   Drizzle schema, client, repositories
  sources/              connector contract, registry/fan-out, connectors
  search/               query parser, filters, dedupe, ranking, service
  commands/             contract, registry, router, one file per command
  whatsapp/             Baileys client, message rendering
  http/                 health, status, search API, rate limiting
public/
```

```
  index.html            the web search interface (single file, no build step)
scripts/
  dev-services.sh       rootless local PostgreSQL and Redis
docs/                   architecture, roadmap, legacy analysis, this manual
```

CHAPTER 3

Requirements

Requirement	Version	Purpose
Node.js	$\geq$ 20 (tested 22.22.2)	Runtime
npm	$\geq$ 9 (tested 11.19.0)	Package management
PostgreSQL	$\geq$ 14 (tested 18.6)	Primary datastore
A spare WhatsApp number	—	The bot links its own device

Not required for Phase 1:

- **Redis** — used from Phase 2 for queues. The development script starts one if installed, and the platform runs without it.
- **An AI API key** — the platform runs fully without one.
- **Docker** — not used. Local services run rootless.

Use a dedicated WhatsApp number

Two bots on one number fight over the session and disconnect each other repeatedly. Give this platform a number of its own.

CHAPTER 4

Installation

4.1 Obtain and install

```
git clone <your-repository-url> phc-platform
cd phc-platform
npm install
```

`npm install` pulls roughly 240 packages, most of them the Baileys dependency tree. Several minutes is normal.

> **Skipped install scripts are expected**
>
> npm may report that install scripts were not run for Baileys, protobufjs and esbuild. This is harmless — the platform, its tests and its build all work correctly without them.

4.2 Verify the toolchain

```
npm run typecheck    # tsc --noEmit
npm test             # vitest run (75 tests)
npm run build        # tsc -> dist/
```

All three should complete without output other than their own banner. The 75 unit tests cover pure logic and require neither a database nor a network, so they pass on a machine with nothing else set up.

CHAPTER 5

Configuration

5.1 Where configuration lives

File	Role
`.env`	Your real configuration. Git-ignored, never committed. This is the file you edit.
`.env.example`	The committed template documenting every variable. Copy it when setting up a new machine.
`src/config/env.ts`	The code defining and validating the variables. The only place in the codebase that reads `process.env`.

```
cp .env.example .env
```

5.2 Validation behaviour

Configuration is parsed and validated *once, at startup*, and then frozen. Two consequences matter in practice:

1. A malformed value stops the process immediately with a message naming the offending variable. You find out when you start the service, not when a member sends a command that happens to use it.
2. Changing `.env` requires a restart. `npm run dev` reloads on source changes but not on configuration changes.

5.3 Complete variable reference

Required

Variable	Description
`DATABASE_URL`	PostgreSQL connection string. Must be a valid URL. Example: `postgres://127.0.0.1:55432/phc`

Runtime

Variable	Default	Description
`NODE_ENV`	`development`	`development`, `test` or `production`. Development enables human-readable logs.
`PORT`	`3100`	HTTP port for the web interface and API.
`LOG_LEVEL`	`info`	`trace` through `fatal`.
`TIMEZONE`	`Africa/Nairobi`	Display timezone.

WhatsApp

Variable	Default	Description
`WHATSAPP_PROVIDER`	`baileys`	Only `baileys` is implemented.
`WHATSAPP_AUTH_DIR`	`./auth`	Where session credentials are stored. Git-ignored. **Treat as a secret.**
`WHATSAPP_PAIR_NUMBER`	empty	Digits only, no `+`. Set to pair by code; leave blank for a QR code.
`COMMAND_PREFIX`	`.`	Command prefix, 1 to 3 characters.

Administration

Variable	Default	Description
`ADMIN_NUMBERS`	empty	Comma-separated phone numbers, digits only, no `+`. Validated at startup.

Opportunity sources

Variable	Description
`SOURCES_ENABLED`	Comma-separated connector slugs. Available: `greenhouse`, `remotive`, `jobicy`, `remoteok`, `arbeitnow`, `reliefweb`. Default: `greenhouse,remotive,arbeitnow,jobicy,remoteok`
`GREENHOUSE_BOARDS`	Employer board tokens, one per organisation. Default: the four verified organisations
`RELIEFWEB_APPNAME`	Free registered appname. Blank (the default) keeps ReliefWeb disabled.

AI providers (optional)

Variable	Default	Description
`AI_PROVIDER`	`gemini`	`gemini`, `anthropic`, `openai` or `none`.
`GEMINI_API_KEY`	empty	**Free** key from Google AI Studio.
`GEMINI_MODEL`	`gemini-3.8-flash`	Model identifier.
`ANTHROPIC_API_KEY`	empty	Paid. From the Anthropic Console.
`ANTHROPIC_MODEL`	`claude-opus-5`	Model identifier.
`OPENAI_API_KEY`	empty	Provider not implemented.
`OPENAI_MODEL`	`gpt-4o-mini`	Model identifier.

Web search (optional)

Powers the opportunity fallback. See Chapter 12.

Variable	Default	Description
`WEB_SEARCH_PROVIDER`	`none`	`tavily`, `searxng`, `provider` or `none`.
`TAVILY_API_KEY`	empty	Free-tier key from `https://app.tavily.com/`.
`SEARXNG_URL`	empty	Base URL of a SearXNG instance you host.

5.4 A complete example

```
NODE_ENV=development
PORT=3100
LOG_LEVEL=info
TIMEZONE=Africa/Nairobi
DATABASE_URL=postgres://127.0.0.1:55432/phc
WHATSAPP_PROVIDER=baileys
WHATSAPP_AUTH_DIR=./auth
WHATSAPP_PAIR_NUMBER=
COMMAND_PREFIX=.
ADMIN_NUMBERS=254700000000
AI_PROVIDER=gemini
GEMINI_API_KEY=your-free-key
GEMINI_MODEL=gemini-3.8-flash
WEB_SEARCH_PROVIDER=tavily
TAVILY_API_KEY=tvly-...
SEARXNG_URL=
SOURCES_ENABLED=greenhouse,remotive,arbeitnow,jobicy,remoteok
GREENHOUSE_BOARDS=oneacrefund,resolvetosavelives,dimagi,givedirectly
RELIEFWEB_APPNAME=
```

CHAPTER 6

Database

6.1 Option A: rootless local server

The bundled script creates a PostgreSQL cluster owned by your user, on a non-default port, inside the project directory. It requires no `sudo` and never touches a system-wide installation.

```
./scripts/dev-services.sh start     # start, create the database
./scripts/dev-services.sh status
./scripts/dev-services.sh stop
./scripts/dev-services.sh destroy   # stop and delete all local data
```

Service	Port	Data directory
PostgreSQL	55432	`.devservices/postgres`
Redis (if installed)	56379	`.devservices`

6.2 Option B: your own PostgreSQL

Point `DATABASE_URL` at it. No other change is needed.

Prefer 127.0.0.1 over localhost

`localhost` can resolve to IPv6 `::1`, which the development cluster does not bind. If a connection fails with `localhost`, use `127.0.0.1`.

6.3 Creating the schema

```
npm run db:push        # apply the schema directly (development)
```

For production, use versioned migrations:

```
npm run db:generate    # write SQL migrations to ./drizzle
npm run db:migrate     # apply them
npm run db:studio      # browse the data
```

6.4 Schema

Eight tables and nine enumerated types.

Table	Purpose
`opportunities`	Every opportunity, of all fourteen types. One table, not fourteen.
`sources`	One row per connector: enablement, trust score, fetch interval, attribution obligation, health.
`members`	WhatsApp JID plus the profile a member chose to provide.
`subscriptions`	A member's interest in a topic.
`saved_opportunities`	Opportunities a member kept.
`command_log`	Command usage for analytics. Never stores raw message text.
`group_settings`	Per-group moderation configuration, created lazily on first change.
`warnings`	Append-only moderation log, so a removal can be justified.

Enum	Values
`opportunity_type`	JOB, INTERNSHIP, SCHOLARSHIP, FELLOWSHIP, RESEARCH, GRANT, CONFERENCE, TRAINING, HACKATHON, VOLUNTEER, PROJECT, MENTORSHIP, EVENT, PROGRAM
`opportunity_status`	DISCOVERED, PENDING_REVIEW, VERIFIED, PUBLISHED, EXPIRED, REJECTED
`verification_status`	UNVERIFIED, SOURCE_VERIFIED, AI_ASSISTED, ADMIN_VERIFIED, EXPIRED, FLAGGED
`experience_level`	UNKNOWN, ENTRY, GRADUATE, MID, SENIOR, EXECUTIVE
`education_level`	UNKNOWN, SECONDARY, CERTIFICATE, DIPLOMA, BACHELORS, MASTERS, DOCTORATE
`category`	20 subject categories, from PUBLIC_HEALTH and EPIDEMIOLOGY through to CYBERSECURITY and OTHER
`notify_mode`	IMMEDIATE, DAILY, WEEKLY, OFF
`source_kind`	API, RSS, SCRAPE, MANUAL
`moderation_action`	OFF, WARN, DELETE, REMOVE

Schema and domain must agree

`drizzle-kit` bundles the schema as CommonJS and cannot resolve a `.js` specifier onto a `.ts` source. The schema therefore declares its enum values directly and imports only *types* from the domain, guarded by a compile-time equality assertion. Add a value to the domain without adding it to the schema and `npm run typecheck` fails — by design.

CHAPTER 7

Running the platform

7.1 Commands

Command	Purpose
`npm run dev`	Development: watch mode, human-readable logs.
`npm run build`	Compile TypeScript to `dist/`.
`npm start`	Production: run the compiled output.
`npm run typecheck`	Type check without emitting.
`npm test`	Run the 133 unit tests.
`npm run check`	Typecheck and test together.
`npm run search`	Search from the terminal (see below).
`npm run ai:check`	Verify the AI provider, key and model really work.
`npm run db:push`	Apply the schema.
`npm run db:generate`	Generate migrations.
`npm run db:migrate`	Apply migrations.
`npm run db:studio`	Browse the database.

7.2 Startup order

Configuration, then database, then HTTP, then WhatsApp. The order is deliberate: connecting to WhatsApp before the database is reachable would leave members talking to a bot that cannot answer. Shutdown reverses the order and flushes pending background writes before closing the connection pool.

7.3 Searching from the terminal

The fastest way to exercise sources, normalisation, de-duplication and ranking without a WhatsApp connection:

```
npm run search -- public health
npm run search -- nairobi internship
npm run search -- --json epidemiology      # machine-readable, scores included
npm run search -- --no-persist remote      # skip database writes
```

It prints the per-source breakdown and the exact message a member would receive. In `--json` mode stdout carries only JSON, so it pipes cleanly; logs go to stderr.

CHAPTER 8

Integration: WhatsApp

8.1 Ownership model

The platform owns its own WhatsApp number and its own session. Authentication state is written to `WHATSAPP_AUTH_DIR` on your machine and never leaves it. No third-party pairing website is involved, and no session string is pasted into an environment variable.

8.2 Pairing with a QR code

1. Leave `WHATSAPP_PAIR_NUMBER` blank.
2. Run `npm run dev`.
3. A QR code is printed in the terminal.
4. On the phone: **WhatsApp** → **Settings** → **Linked devices** → **Link a device**, then scan.

QR codes rotate roughly every twenty seconds; a fresh one is printed automatically.

On a server run with `./ctl.sh` (see `docs/DEPLOY-VPS.md`), `./ctl.sh pair` draws the same QR code in the terminal and closes by itself once the phone has linked. The QR is never shown on a web page unless `PAIRING_PAGE=on` is set, which is meant only for shared hosting with no terminal.

8.3 Pairing with an eight-character code

1. Set `WHATSAPP_PAIR_NUMBER` to the number in international format, digits only, no `+`.
2. Run `npm run dev`.
3. The code is logged after a few seconds.
4. On the phone: **Linked devices** → **Link with phone number instead**, then enter it.

8.4 Confirming the connection

```
curl -s http://127.0.0.1:3100/health
# {"status":"ok","whatsapp":"connected","uptimeSeconds":12}
```

Session credentials persist across restarts. To unlink, stop the process, delete `WHATSAPP_AUTH_DIR`, and pair again.

8.5 Who can use the bot

Any number, including the bot's own. Every command is open to every sender; only commands marked `adminOnly` check `ADMIN_NUMBERS`, and group commands check that chat's own admin list.

That includes the number the bot is paired to. If it is paired to the operator's phone, the operator can drive it from that phone. Earlier builds discarded `fromMe` messages, so the bot connected normally and then silently ignored its own owner; it now processes them, with a loop guard that remembers sent message identifiers so an echo of a reply is never re-handled as input.

Members on newer LID-based WhatsApp accounts are identified by the phone-number JID that WhatsApp supplies alongside the LID (`participantPn` / `senderPn`). Without that, such a member's identity could never match `ADMIN_NUMBERS`, and administrators on those accounts would find admin commands silently failing.

8.6 Reconnection and automatic re-pairing

Ordinary disconnections are retried with exponential backoff, capped at sixty seconds.

A *logged out* session is handled differently. Those credentials are already dead — WhatsApp has invalidated them and nothing can revive them — so the platform deletes them and starts pairing again by itself. A fresh QR code, or a pairing code when `WHATSAPP_PAIR_NUMBER` is set, appears in the logs.

Re-pairing is capped at three attempts per hour. Beyond that the platform stops and says so, rather than spinning indefinitely generating pairing requests against WhatsApp. A successful connection resets the counter.

Why deleting the credentials is safe

It only happens on an explicit logout, where the stored session can no longer authenticate anything. Retaining it would make every subsequent reconnect fail in exactly the same way, leaving the bot dark until a human noticed.

CHAPTER 9

Integration: the web interface

9.1 Using it

Start the platform and open `http://localhost:3100`. No account, no login, no WhatsApp. A student can find an opportunity without joining anything first.

The interface is a single static file with no build step. It is mobile-first, supports light and dark themes, and renders all text through `textContent` rather than `innerHTML`, because titles and organisation names come from third-party feeds.

9.2 HTTP endpoints

Route	Purpose
`GET /`	The search interface.
`GET /api/search`	Search. Parameters: `q`, `location`, `type`, `remote`, `page`, `pageSize`.
`GET /api/sources`	Which sources are active.
`GET /api/filters`	Opportunity types, for the filter controls.
`GET /health`	Liveness probe. 200 when WhatsApp is connected, 503 otherwise.
`GET /api/status`	Operational status. Contains no personal data.

9.3 Example

```
curl -s "http://127.0.0.1:3100/api/search?q=public+health&pageSize=4"
```


```
{
  "results": [
    {
      "title": "Technical Advisor, Surveillance, Data Use and Decision Science",
      "organization": "Resolve To Save Lives",
      "summary": "...",
      "type": "JOB",
      "typeLabel": "Job",
      "categories": ["Epidemiology", "Global Health", "Health Informatics"],
      "location": "Kinshasa, DRC",
      "country": "CD",
      "remote": null,
      "deadline": null,
      "daysLeft": null,
      "applicationUrl": "https://job-boards.greenhouse.io/...",
      "sourceName": "Employer career boards",
      "verified": false,
      "score": 63.6
    }
  ],
  "total": 4,
  "page": 1,
  "pageSize": 4,
  "hasMore": false,
  "degraded": false,
  "unavailableSources": [],
  "attributions": [],
```

```
  "durationMs": 3768,
  "interpreted": { "terms": "public health", "type": null, "location": null }
}
```

The `interpreted` block reports how the query was parsed, and `degraded` together with `unavailableSources` lets a client state that results may be incomplete rather than implying that nothing exists.

9.4 Rate limiting

Endpoint group	Limit	Window
`/api/search`	30 requests	5 minutes per IP
`/api/sources`, `/api/filters`	120 requests	5 minutes per IP

This is not arbitrary caution. Every search fans out to upstreams whose terms cap how often they may be polled; an unlimited public endpoint would exhaust a shared quota and risk the platform's access for every user. Exceeding the limit returns HTTP 429 with a `retry-after` header.

9.5 Security headers

Every response carries `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer` and a restrictive Content Security Policy with no third-party script or style origins. Outbound application links use `rel="noopener noreferrer"`.

CHAPTER 10

Integration: opportunity sources

10.1 How sources work

Each connector is independently enabled, isolated behind its own twelve-second timeout, rate-limited to its upstream's stated policy, and cached. One failing source degrades that source alone, and the user is told the list may be incomplete.

The result cache's time-to-live *is* the connector's minimum fetch interval, so a polite source is polled politely without any caller having to remember to.

10.2 Registered sources

Six connectors, five active by default.

Source	Slug	Trust	Interval	Notes
Employer career boards	`greenhouse`	85	3 h	First-party postings straight from the employer. Keyless. Organisations are configured, not hard-coded.
Remotive	`remotive`	60	6 h	Remote roles. Attribution and link-back required by their terms.
Jobicy	`jobicy`	45	3 h	Remote board with an industry filter. Keyless.
RemoteOK	`remoteok`	45	6 h	Remote board. Keyless. Attribution required.
Arbeitnow	`arbeitnow`	40	3 h	Keyless breadth. Heavily German/European, no server-side search.
ReliefWeb	`reliefweb`	90	2 h	UN, NGO and humanitarian vacancies. **Disabled** pending an approved appname.

An ambitious fetch that never finishes is worth nothing

Arbeitnow originally pulled two pages of several megabytes each and blew the twelve-second connector timeout on *every* search, contributing nothing for the entire time it was enabled. It now fetches one page and returns reliably in about six seconds. For a low-trust breadth source that is the right trade — and the general lesson is that a source which times out is worse than a smaller one that succeeds, because it also costs the member the wait.

10.3 Employer career boards (recommended)

This is the highest-signal source available, and it works differently from an aggregator: instead of one upstream returning everything, you name the organisations you care about and the platform reads their own career boards. Postings are authoritative, carry real application deadlines where the employer set one, use the employer's own department labels as a classification signal, and link directly to the employer's site.

Four organisations are verified and enabled by default: `oneacrefund`, `resolvetosavelives`, `dimagi`, `givedirectly`.

To check whether another organisation publishes a public board:

```
curl -s "https://boards-api.greenhouse.io/v1/boards/<token>/jobs" | head -c 200
```

A 200 response containing a `jobs` array means it works. Add the token to `GREENHOUSE_BOARDS` and restart. The token is usually the organisation's name in lower case. A board that is renamed or withdrawn is logged and skipped; the other organisations are unaffected.

The cheapest way to improve results

Every health employer you add is authoritative, keyless, and ranks above the aggregators. This has a larger effect on result quality than any other change available to an administrator.

10.4 Enabling ReliefWeb

ReliefWeb is where WHO, UNICEF, UNFPA, MSF, IRC and national health ministries post, with real closing dates. It is the single highest-value source for this community.

Since 1 November 2025 the appname must be **pre-approved**; one cannot simply be invented.

1. Complete the request form linked from https://apidoc.reliefweb.int/parameters#appname
2. Choose a name in their required form: organisation + purpose + random characters, for example `phc-colosseum-opportunities-7f3a`
3. Wait for ReliefWeb's email — requests are reviewed by a person.
4. Set `RELIEFWEB_APPNAME` and add `reliefweb` to `SOURCES_ENABLED`.

This connector is unverified

The ReliefWeb connector follows the published v2 API documentation but has **not** been exercised against the live endpoint, because the project has no approved appname. ReliefWeb decommissioned v1, and v2 returns HTTP 403 to unregistered callers. Treat its field mapping as unverified until your first successful synchronisation, and check `.sources` and the logs afterwards.

10.5 On LinkedIn and similar platforms

LinkedIn has no public jobs-search API. Its Talent Solutions partner APIs are for *posting* jobs, are gated behind partner approval, and do not expose job search. Its User Agreement prohibits scraping, and that prohibition is enforced: *hiQ Labs v. LinkedIn* ran for six years and ended with LinkedIn prevailing on breach of contract in 2022; hiQ ceased operations. Scraping also attracts IP blocking and, when performed with an account, account termination.

The platform therefore contains no LinkedIn scraper, and adding one is not recommended.

Legitimate routes to overlapping coverage:

Provider	Notes
JSearch (RapidAPI)	Resells the Google for Jobs index, which includes LinkedIn postings. Documented API, free tier, dependable terms.
Adzuna	Free API, around twenty countries.
Jooble	Free API key.
Careerjet	Free affiliate identifier.

Each is a connector of roughly one hundred lines. If you obtain genuine LinkedIn partner access, the connector architecture accepts it like any other source.

10.6 Adding a source

Create a file under `src/sources/<kind>/` exporting a `SourceConnector`:

```
import type { ConnectorMeta, FetchContext, SourceConnector } from '../types.js';
import { buildUrl, getJson } from '../common/http.js';

const meta: ConnectorMeta = {
  slug: 'who-careers',
  name: 'WHO Careers',
  kind: 'API',
  homepageUrl: 'https://careers.who.int',
  supplies: ['JOB', 'INTERNSHIP', 'FELLOWSHIP'],
  minFetchIntervalMinutes: 120,  // honour the upstream's stated limit
  trustScore: 95,                // first-party postings rank higher
  attributionRequired: false,
  unavailableReason: null,       // or a string when a key is missing
};

export const whoConnector: SourceConnector = {
  meta,
  async fetch({ query, signal, log }: FetchContext) {
    const payload = await getJson<Response>(
      buildUrl(ENDPOINT, { q: query.terms }), { signal },
    );
    return payload.items.map((item) => ({ title: item.title /* ... */ }));
  },
};
```

Then register it in `ALL_CONNECTORS` in `src/sources/registry.ts` and add the slug to `SOURCES_ENABLED`.

A connector returns raw records and maps only what its upstream actually said. It never classifies, de-duplicates or formats — normalisation does that identically for every source. Always use `getJson` rather than `fetch` directly, so the connector inherits timeouts, retry policy, the response-size ceiling and the identifying user agent.

Set `unavailableReason` when a key or registration is missing: the connector then stays registered and explains itself instead of being silently absent.

CHAPTER 11

Integration: AI providers

11.1 What AI is used for

Two features depend on a model:

- `.ask` — general public-health question answering.
- The **opportunity web fallback** — when every configured source returns nothing, the platform searches the live web and extracts real opportunities from the results (Chapter 12).

Everything else — search, filtering, classification, de-duplication, ranking, group administration — works with no provider at all. Rule-based classification is the permanent floor, not a placeholder, so a missing, failing or unbilled provider never degrades the platform.

11.2 Gemini — the default

Google AI Studio issues a **free** API key with a usable free tier. That is why it is the default: the assistant and the fallback both work without a billing account.

```
AI_PROVIDER=gemini
GEMINI_API_KEY=your-key-here
GEMINI_MODEL=gemini-3.8-flash
```

Get a key at https://aistudio.google.com/apikey, then verify it:

```
npm run ai:check
```

> **Always run ai:check after changing a key or model**
>
> The two things most likely to be wrong — an unusable model name and a key without the right tier — surface only on a real call. The diagnostic makes one plain call and one search call and reports exactly which failed. Finding out in one second beats finding out from a member whose question silently failed.

Two Gemini behaviours worth knowing

Models are retired without notice. `gemini-2.5-flash` returns HTTP 404 for accounts created after its deprecation, with the real reason only in the response body. If `ai:check` reports the model is unavailable, list what your key can actually use:

```
curl -s "https://generativelanguage.googleapis.com/v1beta/models?key=$GEMINI_API_KEY" \
  | grep -o '"name": "models/[^"]*"'
```

Thinking tokens come out of the output budget. Gemini's flash models reason before answering, and that reasoning is charged against `maxOutputTokens`. A small budget is consumed entirely by thinking and the call returns a valid response with no text — indistinguishable from a failure. The provider enforces a floor of 1024 tokens so this cannot happen.

11.3 Anthropic

Paid, and higher quality for difficult reasoning.

```
AI_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-opus-5
```

A consumer subscription is not API access

Anthropic API access is a separate paid product. A consumer Claude Pro or Max subscription does **not** include an API key and does **not** grant API access. Nor can the credentials of a Claude desktop or CLI session be repurposed to run a service: they are scoped to that application and expire with it. Obtain a key from the Anthropic Console (https://console.anthropic.com/); it is billed per token, independently of any chat subscription.

Indicative rates per million tokens: Claude Opus 5 $5 in / $25 out, Claude Sonnet 5 $2 / $10, Claude Haiku 4.5 $1 / $5.

11.4 OpenAI

`AI_PROVIDER=openai` is accepted by configuration but the provider is not implemented. It logs a clear warning and leaves AI disabled, rather than failing obscurely at the first question.

11.5 Switching providers

Two lines in `.env`. Only `src/ai/gemini/provider.ts` and `src/ai/anthropic/provider.ts` import a vendor SDK; every other file depends on the `AiProvider` interface. No command, service or test changes when the provider does.

11.6 Guardrails

Each is covered by a test.

- **No individual medical advice.** A public-health community attracts clinical questions. The assistant answers general knowledge and declines to advise on any individual's care.
- **No invented opportunities.** It will not fabricate a scholarship, deadline, link or organisation. A model guessing a deadline costs a member a real application.
- **Member text is data**, wrapped as a quoted question, never treated as instructions.
- **A separate budget guard** — 15 questions per member per hour, independent of the general rate limit, so one person cannot spend the community's tokens.
- **Errors never leak internals.** Provider messages may name a key or an endpoint; the member-facing text never repeats them.
- **Transient failures are retried.** Gemini returns HTTP 503 "high demand" often enough that a single attempt makes the feature look broken. Retryable failures get three attempts with backoff; an authentication error or a bad model name fails immediately.

11.7 The .ask command

One command, not nine. Separate `.gpt`, `.claude` and `.gemini` commands would differ only in which company bills the operator, and would hard-code vendor choice into the command surface — precisely what this layer exists to avoid. Which model answers is an operator decision in `.env`; a member does not need to know.

CHAPTER 12

Integration: web search

12.1 Why search is separate from the model

The fallback originally used the model's own built-in search — Anthropic's server-side tool, Google's Search grounding. Both turned out to be **paid** features. On a free Gemini key, grounded search returns HTTP 429 `RESOURCE_EXHAUSTED` on every call while plain generation works perfectly well.

So the two were separated. Any search backend can supply the pages; any model can read them:

```
web search backend  ->  real URLs + snippets  ->  model extracts  ->  opportunities
     (free)                                         (free tier)
```

This also makes the model's job narrower and more honest. It no longer searches or recalls; it extracts from numbered results it was handed, and every URL it reports can be checked against that list.

12.2 Choosing a backend

Setting	Backend	Cost
`tavily`	Tavily, built for AI agents. Key from https://app.tavily.com/	free tier
`searxng`	A SearXNG instance you host. No key, no quota, no vendor	free
`provider`	The AI provider's own grounded search	billed by them
`none`	No web fallback	—

Tavily

```
WEB_SEARCH_PROVIDER=tavily
TAVILY_API_KEY=tvly-...
```

SearXNG

The zero-cost-forever option: you run it, so nobody can price you out of your own fallback.

```
docker run -d -p 8080:8080 \
  -e SEARXNG_BASE_URL=http://localhost:8080/ searxng/searxng
```

Enable the JSON API in that instance's `settings.yml`:

```
search:
  formats: [html, json]
```

```
WEB_SEARCH_PROVIDER=searxng
SEARXNG_URL=http://localhost:8080
```

Public SearXNG instances are not a supported target

Most serve a captcha to automated callers, and pointing a community bot at someone else's free instance is not reasonable use. Host your own.

12.3 How the fallback behaves

- **Only on zero results.** It never runs when the sources already answered, so a normal search costs nothing.
- **Results go through the same pipeline** — normalised, filtered, de-duplicated, ranked — so they render identically.
- **Marked `AI_ASSISTED`**, and the reply says plainly that these came from a web search and are not verified.
- **Every entry must carry a real URL** copied from the search results. Entries without one are discarded: a lead a member cannot click through to verify looks actionable and is not.
- **No invented posting dates.** The model has no reliable one, and guessing would corrupt both the recency filter and the ranking.
- **Charged against the member's AI budget** — the same 15/hour guard `.ask` uses.

Filtering is deliberately lenient here

Fallback results are filtered with `treatUnknownAsMatch`, and the reason matters. Web pages rarely state an ISO country code, and such a result never carries a posting date. Applying the strict source filters discarded every opportunity the fallback found — two genuine Kenyan scholarships, thrown away for lacking fields a web page does not have.

Leniency applies to *unknowns only*. A stated country that does not match is still rejected, an expired deadline is still rejected, and an explicit remote requirement is still enforced. Five regression tests pin that distinction.

12.4 Verified example

With `WEB_SEARCH_PROVIDER=tavily` and `AI_PROVIDER=gemini`, both on free tiers:

```
.job public health scholarship kenya

1. Jill Sanders Memorial Scholarship 2026 - HTAi
   Deadline: 15 Oct 2026 (11 days)
2. KMTC Fully Funded Scholarships for Specialised Health Training
   Kenya Medical Training College

.job global health fellowship africa

1. TIMERA Fellowship 2026-2029 - Epidemic Modelling for African Public Health
   Deadline: 15 Oct 2026 (11 days)
2. FOR-GHP Fellowship 2027-2028 - German Federal Ministry of Health
   Deadline: 21 Oct 2026 (17 days)
3. Healthy Futures South Africa PhD Fellowship - University of Cape Town
   Deadline: 31 Oct 2026 (27 days)
```

Real deadlines, real links, flagged as unverified. Total running cost: nothing.

CHAPTER 13

Command reference

All commands begin with `.` by default. Thirty-four commands, sixty-three names including aliases.

13.1 Opportunities

Command	Aliases	Description
`.job <keywords>`		Search by keyword
`.jobs <filters>`		Search by location, type or recency
`.scholarships`	`.scholarship`	Scholarships and tuition funding
`.internships`	`.internship`	Internships and attachments
`.fellowships`	`.fellowship`	Fellowships
`.research`	`.projects`	Research, PhD and postdoctoral posts
`.grants`	`.grant`, `.funding`	Grants and calls for proposals
`.events`	`.event`	Events, webinars, summits
`.conferences`	`.conference`	Conferences and calls for abstracts
`.training`	`.trainings`, `.courses`	Short courses and workshops

13.2 Navigating results

Command	Aliases	Description
`.next`	`.more`	Next page
`.prev`	`.previous`, `.back`	Previous page
`.details <n>`	`.detail`, `.info`	Full description of result n

13.3 Group management

Admin-only. Authorisation is checked against the *group's* own admin list on every call, never inferred from the command name. Participant actions additionally require the bot itself to be an admin, and it refuses to act on itself — removing or demoting itself would leave the group unmanaged with no way back.

Command	Aliases	Description
`.groupinfo`	`.ginfo`	Details and moderation settings (any member)
`.add <number>`		Add someone to the group
`.kick @user`	`.remove`	Remove someone (refuses to remove an admin)
`.promote @user`		Make someone a group admin
`.demote @user`		Remove someone as group admin
`.open` / `.close`	`.unmute` / `.mute`	Who may send messages
`.lock` / `.unlock`		Who may edit group info
`.welcome`		Greet people who join: `on`, `off`, `set <message>`, `test`, `reset`
`.warn @user`		Issue a warning, with an optional reason
`.warnings @user`	`.warns`	Active warnings, with reasons and who issued them
`.resetwarns @user`	`.clearwarns`	Clear warnings

Three design decisions are worth knowing:

- **Targets come from real WhatsApp mentions**, read from the message's context information, not from parsing text that looks like `@2547...`. The visible handle is only a label, so trusting it would let anyone forge a target.
- **Warnings are an append-only log**, not a counter. Clearing them deactivates rather than deletes, so an administrator asked why a member was removed can answer with evidence.
- **Nobody is removed automatically unless an administrator chose it**, by setting a moderation rule to `remove` (see below). Otherwise reaching the warning limit only tells an administrator, who decides.

13.4 Moderation

The bot reads every message in a group and enforces that group's rules. **Group admins, platform administrators and the bot itself are never moderated.** Deleting and removing need the bot to be a group admin; without that it can only warn, and says so once an hour rather than on every message.

Command	Description
`.mod`	Overview of this group's rules (alias `.moderation`)
`.mod on`	Recommended protection: delete and warn for links, adverts and spam
`.mod strict`	The same, and remove at the warning limit
`.mod off`	Every rule off
`.mod test <message>`	What a message would trigger, without deleting anything
`.mod copy`	Copy these rules to the other groups of the WhatsApp community the caller administers
`.antilink <action> [invites/all]`	`invites` (default): other WhatsApp groups and channels, `wa.me`, Telegram, Discord. Job links still work. `all`: every link
`.antilink allow <domain>`	Never touch links to a domain, such as `who.int` (`unallow` removes it)
`.antipromo <action>`	Adverts, sales pitches, money schemes, follower boosting, "join my channel" (alias `.antiad`)
`.antipromo ai on/off`	Let the AI decide borderline posts the rules cannot
`.antispam <action>`	More than 6 messages in 10 seconds, the same message 3 times, or the same message in 3 groups within 10 minutes
`.badword add <word>`	Ban a word or phrase; `del`, `list` and `clear` manage the list
`.badword <action>`	What to do about banned words
`.silence @user [30m/2h/1d]`	Delete everything a member posts for a while: one hour by default, up to 30 days
`.unsilence @user`	Lift a silence; `.silenced` lists them
`.warnlimit <1-10>`	Warnings before `remove` rules remove someone

Each `<action>` is one of:

- `warn` — warn the sender;
- `delete` — delete the message and warn (`on` means this);
- `remove` — delete, warn, and remove the sender at the warning limit;

- `off`.

How it behaves:

- **Tuned for an opportunities community.** Posts that mention scholarships, vacancies, deadlines or stipends get the benefit of the doubt, so an application link or a stipend amount alone never counts as an advert. `.mod test` shows the score and the signals behind it before a rule is turned on.
- **One offence per minute.** A burst of rule-breaking earns one warning per rule per minute, while every offending message is still deleted, so one flood does not use up all of a member's warnings and the bot never floods the group with its own notices.
- **The AI is a second opinion, never the first.** Only posts the rules already find suspicious reach it, only in groups that turned it on, within 60 reviews an hour. An AI error leaves the post alone.
- **Either identity works.** WhatsApp lists members of some groups by an anonymous id rather than their number. Admin checks, silences and warnings match members by both.

Settings live in the database and take effect on the next message. The tables are added automatically the first time the new version starts.

13.5 Welcome messages

`.welcome on` greets everyone who joins a group: added by an admin, through an invite link, by an approved request, or from the community. In a custom message set with `.welcome set`, `{name}` tags the newcomers and `{group}` is replaced with the group's name; without one, a standard message introduces the community and points to `.menu`. `.welcome test` previews it, tagging the admin who asked, and `.welcome reset` returns to the standard message.

- **One welcome per wave.** People who join within a few seconds of each other are welcomed in a single message tagging all of them (up to twenty, then "and N others"), and a group gets at most one welcome a minute. A shared invite link never turns the bot into a flood.
- **No late welcomes.** Joins that happened while the bot was offline for more than ten minutes are skipped.
- **The bot must be able to post.** In a group closed to admins (`.close`), make it an admin.

13.6 System

Command	Aliases	Description
`.help [command]`	`.h`, `.commands`	Command list, or help for one command
`.menu`		Main menu
`.start`		Getting started
`.about`		About the Colosseum
`.sources`	`.source`	Which sources are searched
`.ping`	`.alive`, `.uptime`	Liveness, uptime, memory

13.7 Locations

Several locations can be given at once, separated by commas or spaces:

```
.job public health nairobi, kisumu
.job public health nairobi, kampala
.scholarships kenya, uganda, tanzania
```

Countries are matched as **OR**. Named cities additionally give a ranking boost rather than acting as a hard filter: someone asking for Nairobi still sees a listing that only says “Kenya”, just below the ones that say Nairobi.

Why this is a list and not one value

The query originally held a single country, resolved from the whole location string. The moment two places were named it resolved to nothing and *silently dropped the location filter altogether*, so `nairobi, kampala` quietly searched the entire world. A list makes the failure impossible.

13.8 Filters

Filters are recognised anywhere in the arguments and combine freely.

Kind	Accepted	Example
Location	Country or city names	`.jobs nairobi`
Region	`africa`, `worldwide`, `global`	`.jobs africa`
Remote	`remote`, `work from home`	`.jobs remote`
Type	`internship`, `scholarship`, `fellowship`, `research`, `grant`, `conference`, `training`, `volunteer`, `hackathon`, `event`, `job`	`.jobs internship`
Recency	`today`, `latest`, `new`, `recent`, `this week`, `month`	`.jobs today`
Deadline	`deadline`, `closing soon`	`.jobs closing soon`
Experience	`entry-level`, `junior`, `graduate`, `trainee`	`.jobs entry-level`

Filters stack: `.jobs nairobi internship today`.

Recognised filter words are removed from the free-text search, so `.jobs nairobi` does not additionally require the word “nairobi” in the title.

CHAPTER 14

The opportunity pipeline

14.1 Normalisation

Every source passes through one funnel, so storage, search, ranking and rendering only ever see one shape. Descriptions are stripped of HTML, with `<script>` and `<style>` bodies removed first. URLs are validated: only absolute `http` and `https` are accepted, so `javascript:`, `data:` and `file:` never reach a user. Dates outside the years 2000 to 2100 are rejected as placeholders.

> **Entity-encoded HTML**
>
> Some sources deliver HTML that is itself entity-encoded. Stripping tags before decoding leaves markup visible as text; decoding before stripping would turn a literal “a < b > c” into deleted content. The implementation strips, decodes, and re-strips only when the result genuinely looks like markup.

14.2 Classification

Rule-based, running with or without an AI provider. Each classifier reads only the text where its signal actually lives — pointing all of them at the full description produced nonsense, such as a vacancy mentioning “present at international conferences” being typed as a conference.

Classifier	Reads	Rationale
Type	title, tags	These name what a posting *is*.
Categories	title and tags (authoritative); body requires two or more matches	One passing mention of "mental health" in a benefits list is not a subject signal.
Experience	title, tags	"Senior stakeholders" in a body is not a seniority signal.
Education	title, body	Qualifications live in the requirements section.
Remote	title, location, body	Remote arrangements are often stated only in prose.
Country	title, location	Resolved only when exactly one country is named.

14.3 De-duplication

Two passes. First, an exact content fingerprint over canonical title, organisation and type. When the organisation is unknown, the source slug is mixed in, so two unrelated employers who both omitted their name do not collapse into one record.

Second, near-duplicates within the same organisation and type: one title being the other plus a qualifier such as "(Remote)" or a duty station. Where the organisation is unknown, no merging occurs — two employers can advertise the same title, and wrongly hiding one costs a member a real opportunity.

The surviving copy is chosen by source trust first, then completeness.

14.4 Ranking

Component	Weight
Relevance	40%
Recency	20%
Verification and source trust	20%
Deadline urgency	10%
Member profile fit	10%

Weights live in one object and can be tuned without touching scoring logic. Deadline urgency peaks for deadlines three to twenty-one days away — far enough to apply, close enough to matter.

Relevance for multi-term queries is phrase-first:

Match	Score
Phrase in title	1.00
All terms in title	0.90
Phrase in tags	0.75
Phrase in body	0.60
All terms in title or tags	0.55
Scattered partial	coverage × 0.2

A quality gate of 0.25 applies when the user supplied terms. Several upstreams ignore or loosely honour their own search parameter, so without a gate they flood the list. Fewer, truer results is the correct trade for a community where a bad lead costs someone a real application.

14.5 Verification lifecycle

```
DISCOVERED --> PENDING_REVIEW --> VERIFIED --> PUBLISHED --> EXPIRED
                    \--------------> REJECTED
```

Everything enters as DISCOVERED and UNVERIFIED. Promotion is a separate, reviewed step, never a side effect of ingestion. The upsert path deliberately does not write status, verification status, verification time or review notes: those record human decisions, and an automated synchronisation overwriting them would silently re-approve content behind an administrator's back.

CHAPTER 15

Security and privacy

- **Secrets only in the environment.** No key is committed; `.env` and `auth/` are git-ignored.
- **Minimal personal data.** A member record holds a WhatsApp JID plus whatever the member chose to provide. No push names, profile photographs or contact-book entries are copied.
- **No personal data in logs.** JIDs are redacted to the form `254***060@s.whatsapp.net`. Raw message text is never logged; only the resolved command name is.
- **Anonymous HTTP surface.** `/api/status` exposes no phone number or member identifier.
- **Administrator authorisation** is checked against `ADMIN_NUMBERS` on the verified sender, before a handler runs. Administrator commands return the unknown-command reply to non-administrators, so their existence is not confirmed.
- **Persisted rate limiting** per member, so restarting cannot reset a flood. Public HTTP endpoints are limited per IP.
- **URL validation and HTML sanitisation** on everything originating from a source.
- **No remote code and no remote control.** The platform never downloads code at runtime and never takes instructions from a remote feed. Everything it runs is in the repository and reviewable.
- **Failure isolation.** A failing source, a thrown handler or an unhandled rejection cannot take the service down.

CHAPTER 16

Development

16.1 Adding a command

Create a file under `src/commands/<category>/`:

```
import type { Command } from '../types.js';

export const profileCommand: Command = {
  name: 'profile',
  aliases: ['me'],
  category: 'members',
  summary: 'View or edit your opportunity profile',
  usage: '.profile',
  examples: ['.profile'],
  // adminOnly: true,            // authorised against ADMIN_NUMBERS
  // exemptFromRateLimit: true,  // only for trivial commands
  async handler(context) {
    await context.reply(`Country: ${context.member.country ?? 'not set'}`);
  },
};
```

Then import and list it in `src/commands/registry.ts`. That is all: `.help` is generated from the registry, so documentation cannot drift, and name or alias collisions throw at startup rather than silently shadowing.

The context provides `reply`, `react`, `args`, `argv`, `member`, `isAdmin`, `log` and session accessors. A handler never touches the WhatsApp socket, so it is unit-testable with no connection.

For a search-backed command, delegate to the shared pipeline:

```
import { runSearch } from '../opportunities/run-search.js';

async handler(context) {
  await runSearch(context, {
    forcedTypes: ['SCHOLARSHIP'],
    commandLabel: '.scholarships',
  });
}
```

16.2 Adding a source

See Chapter 10.

16.3 Testing

```
npm test             # 133 unit tests
npm run test:watch
npm run check        # typecheck plus tests
```

The 133 unit tests cover pure logic and require neither a database nor a network. Test configuration supplies the environment variables that startup validation requires.

CHAPTER 17

Troubleshooting

Cannot reach the database

```
./scripts/dev-services.sh start
npm run db:push
./scripts/dev-services.sh status
```

If `localhost` fails, use `127.0.0.1`: `localhost` can resolve to IPv6, which the development cluster does not bind.

Invalid environment configuration

The message names each offending variable. Common causes: `DATABASE_URL` missing or not a URL; `ADMIN_NUMBERS` or `WHATSAPP_PAIR_NUMBER` containing `+`, spaces or dashes (digits only).

WhatsApp will not connect

Symptom	Cause and remedy
No QR code appears	`WHATSAPP_PAIR_NUMBER` is set. Blank it for QR mode.
QR expires	They rotate every twenty seconds; a new one is printed automatically.
"Session logged out"	The account unlinked this device. Stop, delete `WHATSAPP_AUTH_DIR`, pair again. Not retried by design.
Connects then drops	Another bot is using the same number. Give the platform its own.
Pairing code rejected	Digits only, international format, no `+`.

Few or no results

A search returning nothing is usually a source-coverage gap, not a defect. Check `.sources` or `/api/sources` to see what was actually searched. The aggregators carry little public-health or African content; result quality comes from employer career boards and ReliefWeb. See Chapter 10.

A source keeps failing

Failures are logged at warning level because an unreachable upstream is operational noise, not a platform fault. Searches continue with the remaining sources and the reply states that the list may be incomplete. Do not lower a connector's minimum fetch interval below the upstream's stated policy; Remotive blocks excessive requests.

Port already in use

```
ss -ltnp | grep 3100
```

Change `PORT`, or stop the process holding it. Note that a development server started with `tsx` does not match a process search for `dist/index.js`.

db:push fails with constraint errors on PostgreSQL 18

PostgreSQL 18 gives NOT NULL constraints real names (`members_id_not_null`). drizzle-kit 0.30 does not recognise them, so its diff tries to `DROP` them and fails with `42P16` on any constraint backing a primary key — possibly after having already created new tables.

Apply schema changes with explicit SQL instead:

```
psql "$DATABASE_URL" -f drizzle/0001_group_moderation.sql
```

Migrations in `drizzle/` are written to be idempotent (`IF NOT EXISTS`), so re-running one is safe. On PostgreSQL 17 or earlier, `db:push` works normally.

db:push cannot resolve a module

`drizzle-kit` bundles the schema as CommonJS and cannot resolve a `.js` specifier onto a `.ts` file. The schema therefore declares enum values directly and imports only types, guarded by a compile-time assertion. If you add a value to the domain, add it to the schema too; `npm run typecheck` will tell you if you forget.

CHAPTER 18

Roadmap

Release 1.0 “Neo” is complete. The database schema already carries the columns the later work needs, so none of it requires migrating existing data.

18.1 What Neo includes

Area	Delivered
Opportunities	Generalised model across 14 types, normalisation, rule-based classification, two-pass de-duplication, weighted ranking, verification state machine
Sources	Six connectors, isolated and rate-limited, with attribution obligations carried as data
Surfaces	WhatsApp (42 commands) and a public web search page, both on one search engine
Groups	Participant management; enforced moderation of links, adverts, spam and banned words; silencing; welcome messages; append-only warnings
AI	Provider abstraction with Gemini (default) and Anthropic; `.ask`; guardrails and a per-member budget
Web search	Tavily and self-hosted SearXNG backends, plus provider grounding; live fallback when sources find nothing
Platform	PostgreSQL, rate limiting, admin authorisation, structured logging, health endpoints, automatic database migrations, 214 tests

18.2 Next

Theme	Contents
Opportunity engine	Redis and BullMQ, scheduled synchronisation, database-first search so stored opportunities become searchable, expiry sweep, broken-link checking, more connectors
Community intelligence	Profiles, subscriptions, personalised recommendations, notifications, daily and weekly digests, deadline alerts
Administration	React dashboard, authenticated admin API, review queue, source management, analytics, broadcasts
Multi-AI	OpenAI provider, routing by task and cost, fallback chains

Stored opportunities are not yet searchable

Search queries the live sources on every request; the database is written to but never read from for search. Accumulated opportunities are therefore a record, not an index. Database-first search is the first item of the next phase, and it is what makes the store pay for itself.

Multi-tenancy is a sequencing decision, not a later feature

If the product is to become multi-tenant software-as-a-service, a tenant identifier must be added to every table *before* real organisations and real data exist. While the database holds only disposable

records the migration is nearly free; after the first paying community is onboarded it becomes a painful backfill with downtime. The search engine itself is unaffected.

CHAPTER A

Operational reference

A.1 Default ports

Service	Port	Configurable via
Platform HTTP	3100	`PORT`
Development PostgreSQL	55432	`PG_PORT`
Development Redis	56379	`REDIS_PORT`

A.2 Operational limits

Limit	Value	Rationale
Source fetch timeout	12 s	Ceiling on any single upstream call
Search rate limit	30 / 5 min	Per IP; protects shared upstream quotas
Metadata rate limit	120 / 5 min	Per IP
Member token bucket	20 tokens	Refills 4 per minute, persisted
WhatsApp page size	5	Readable on a phone
Web page size	12 (max 30)	
Stored description	4,000 chars	Full text lives at the source
Response size ceiling	12 MB	Per upstream response
Session lifetime	30 minutes	For pagination and detail views
Reconnect backoff	capped at 60 s	Exponential

A.3 Further reading

Document	Contents
`README.md`	Quick-start and day-to-day reference
`docs/ARCHITECTURE.md`	Design decisions and the reasoning behind them
`docs/ROADMAP.md`	Phase detail
`docs/LEGACY-WOLFBOT-ANALYSIS.md`	Analysis of the predecessor system and why this platform was built as a new service